PocketQuake

From a catalog CSV to a relocation notebook in one command

Min-Seong Seo / SGTL, SNU

Version 1.14.1 | Updated August 21, 2026

What is PocketQuake?

A thin orchestrator that takes a list of earthquakes and produces a relocation + focal-mechanism notebook. It glues two specialised submodules:

- **seismoseo/necis-downloader** — KMA NECIS waveform fetcher (Playwright + the FTP staging server).
- **seismoseo/korea-cluster-relocation** — the picking + HypoInverse + HypoDD + SKHASH pipeline.

One command, end to end:

```
./pocketquake.sh ./examples/chungju/chungju_catalog.csv chungju
```

Output: an executed Jupyter notebook with maps, depth sections, and beachball diagrams for every event you fed in.

Workflow at a glance

catalog.csv

$\downarrow$ *scaffold cluster dir under the eq-cycle submodule*

kma_waveforms/ (NECIS download via Playwright)

$\downarrow$ *eq-cycle pipeline*

PhaseNet+ picking $\rightarrow$ HypoInverse $\rightarrow$ ph2dt $\rightarrow$ dt.ct $\rightarrow$ xcorr $\rightarrow$ dt.cc $\rightarrow$ SKHASH

$\downarrow$

03_results_<slug>.ipynb – map, depth sections, beachballs

You write the catalog and run one command. PocketQuake takes care of dispatching to the right waveform source, registering the cluster, and executing the notebook.

Prerequisites

OS	Linux or macOS (bash wrapper; Windows uses WSL)
glibc	$\geq$ 2.28 — Ubuntu 20.04+, RHEL/Rocky 8+, Debian 10+. CentOS 7 / RHEL 7 are below the line (Playwright fails).
Python	3.10+ (pinned by `environment.yml`)
conda	Miniforge / Miniconda / Mambaforge
gcc/gfortran	$\geq$ 9.3 (for the compiled externals)
Disk	~10 GB (conda env + Chromium + waveforms)
Network	HTTPS to `github.com`, `code.usgs.gov`, `conda-forge`, `necis.kma.go.kr`

NECIS account (apply at `https://necis.kma.go.kr`, research / institutional accounts only; approval 1–5 days).
STP account is optional — only for pre-2020 waveforms.

Install [1/2] — Python environment

```
# 1. Clone with submodules
git clone --recurse-submodules https://github.com/seismoseo/PocketQuake.git
cd PocketQuake

# 2. Create the conda env (one command does everything Python-side)
conda env create -f environment.yml
conda activate pocketquake

# 3. Editable installs for PocketQuake itself + the NECIS submodule
pip install -e . -e external/necis-downloader

# 4. Playwright Chromium (one-time, ~200 MB)
playwright install chromium

# 5. Credentials
cp .env.example .env
#  then edit NECIS_USER, NECIS_PASS
```

Install [2/2] — External programs

PocketQuake's default chain calls these from the shell. None are pip-installable. Put each binary on `$PATH`:

- `hyp1.40` — HypoInverse (USGS Fortran)
- `ph2dt` + `hypoDD` — Waldhauser's HypoDD (Fortran)
- `mseed2sac` — IRIS converter (C)
- `libarchive` / `bsdtar` — already in `environment.yml`

Two research clones reachable by env var:

```
git clone https://github.com/AI4EPS/EQNet.git
echo "EQNET_DIR=$PWD/EQNet" >> ~/PocketQuake/.env

git clone https://code.usgs.gov/esc/SKHASH.git
echo "SKHASH_DIR=$PWD/SKHASH/SKHASH" >> ~/PocketQuake/.env
```

Full per-tool source + build instructions: **docs/EXTERNAL_TOOLS.md**. **Pure-Python option (`-python`):** replaces the Fortran `hyp1.40` + `ph2dt`/`hypoDD` with HypoSVI + EikoNet and `relocDD-py` — no Fortran build needed. See the next-but-one slide.

Catalog format

A plain CSV with 10 columns, KST origin times:

```
Year,Month,Day,Hour,Minute,Second,Latitude,Longitude,Magnitude,Depth
2025,2,7,2,35,34,37.14,127.76,3.1,9
2025,2,7,2,54,38,37.14,127.76,1.4,6
2025,2,7,3,49,4,37.14,127.76,1.5,7
2025,2,8,10,13,23,37.14,127.76,1.6,7
```

- Origin time interpreted as KST (UTC + 9 h)
- PocketQuake auto-derives the cluster epicenter (catalog centroid) and the spatial bounds (bbox + 0.2°); override with `-epi`, `-bounds`
- Magnitude is informational — relocation does not use it
- Depth ≥ 0, measured in km

Run the chungju example

```
./pocketquake.sh ./examples/chungju/chungju_catalog.csv chungju
```

The wrapper:

1. scaffolds a cluster dir under external/korea-cluster-relocation/chungju_cluster/
2. downloads NECIS waveforms (Playwright drives the JS portal, polls the staging queue, streams the FTP files)
3. runs PhaseNet+ → HypoInverse → HypoDD → SKHASH
4. builds + executes the results notebook

Useful flags:

- -python — pure-Python relocation backend (no Fortran; next slide)
- -source mixed — per-event STP+NECIS dispatch
- -picker stead — SeisBench PhaseNet (no EQNet needed)
- -mainshock <UTC> — Gwangyang-style mainshock treatment
- -cores N — cap xcorr workers (default 10)

High-rate stations: automatic anti-aliasing

PhaseNet+ runs at **100 Hz**, so stations recorded above that (200 Hz KMA/KG, 250 Hz nodal/geophone arrays) are down-sampled first.

- **The trap:** EQNet's reader down-samples by plain *linear interpolation, no lowpass* — energy above the new 50 Hz Nyquist *folds* onto the P/S onsets, distorting pick times and first-motion polarity.
- **The fix (v1.12.0):** lowpass *before* the picker — demean → taper → **zero-phase** Butterworth at 45 Hz ($0.9\times$ new Nyquist) → resample. Zero-phase keeps onsets on time (a causal filter would bias picks late).
- Fires **only when native rate > 100 Hz**: 100 Hz data passes through byte-identical; up-sampling (40 → 100 Hz) is alias-free and untouched.

Verified on a real 200 Hz KG trace: energy above 45 Hz drops 0.064% → 0.003%.

Pure-Python backend (`--python`)

A Fortran-free relocation chain that drops in for `hyp1.40` + `hypoDD`:

- **HypoSVI + EikoNet** — absolute location by Stein variational inference over a neural-network travel-time field (pretrained weights shipped).
- **relocDD-py** — Katherine Biegel's Python port of `hypoDD` for the double-difference relocation.

One-time setup (no compiler) — clone the three tools, fetch the weights:

```
git clone https://github.com/katie-biegel/relocDD-py.git
git clone https://github.com/Ulvetanna/HypoSVI.git
git clone https://github.com/Ulvetanna/EikoNet.git
cat >> ~/PocketQuake/.env <<ENV
RELOCDD_PY_DIR=$PWD/relocDD-py
HYPOSVI_DIR=$PWD/HypoSVI
EIKONET_DIR=$PWD/EikoNet
ENV
python -m pipeline.core.fetch_eikonet   # pretrained kim1983 + kim2011
```

```
./pocketquake.sh ./examples/chungju/chungju_catalog.csv chungju --python
```

What you see while it runs

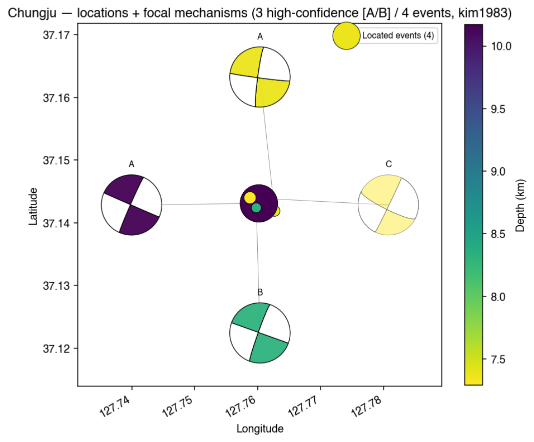
Foreground (`-fg`): full log to terminal.

Default (background): `nohup.out` in the working directory.

```
[pocketquake] scaffolding chungju_cluster ...
[necis] login OK; submitting request for 4 event(s)
[necis] request id 4f3a... status=P -> C, downloading 4 SAC bundles
[eqcycle] picking with phasenet_plus (~30s/event)
[eqcycle] hypoinverse: 4 events located
[eqcycle] xcorr 6 pairs (10 workers) ...
[eqcycle] dt.cc -> hypoDD reloc OK (4 events)
[eqcycle] focal_mechanism: 4 SKHASH solutions
[pocketquake] executing 03_results_chungju.ipynb
```

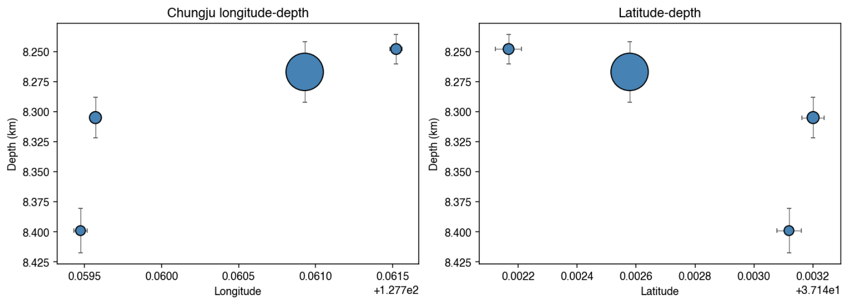
Wall-clock for chungju: ∼15 min on a 16-core box.

Output 1 — Relocated map + focal mechanisms



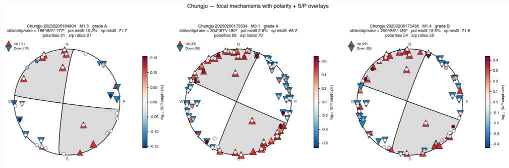
Four located events (depth-coloured dots) at $(37.142^{\circ}\mathrm{N}, 127.760^{\circ}\mathrm{E})$ with quality A/B/C SKHASH beachballs drawn on a leader-line ring. All four solutions converge on a near-vertical N–S right-lateral strike-slip plane.

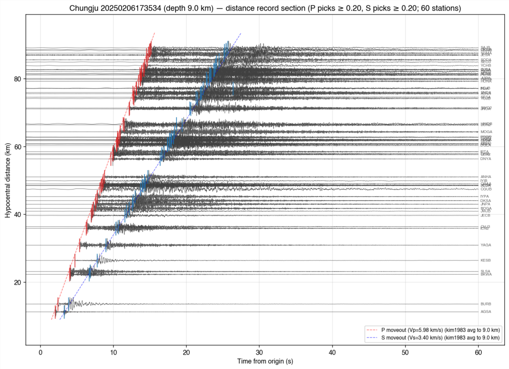
Output 2 — Depth sections



Three orthogonal cross-sections (E–W, N–S, along-strike) showing the depth distribution. HypoDD relative-relocation tightens the cluster from ∼1 km scatter (KMA bulletin) to ∼200 m.

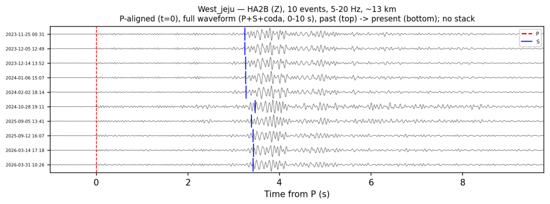
Output 3 — Beachball gallery + record section

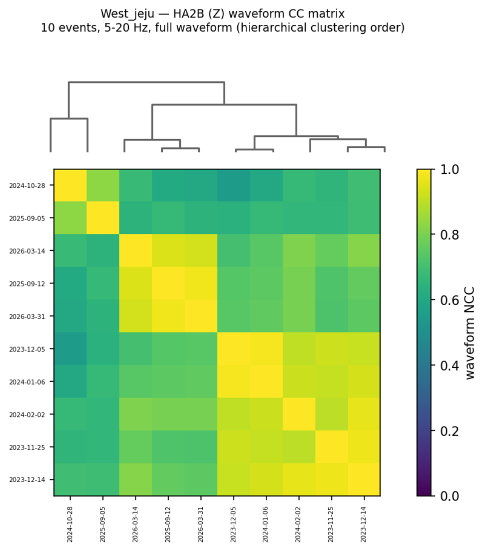




The notebook bundles a per-event beachball gallery (left) and a record-section view of the M3.1 mainshock with PhaseNet+ pick markers, first-motion polarities, and the SKHASH-inferred S/P amplitude ratios (right).

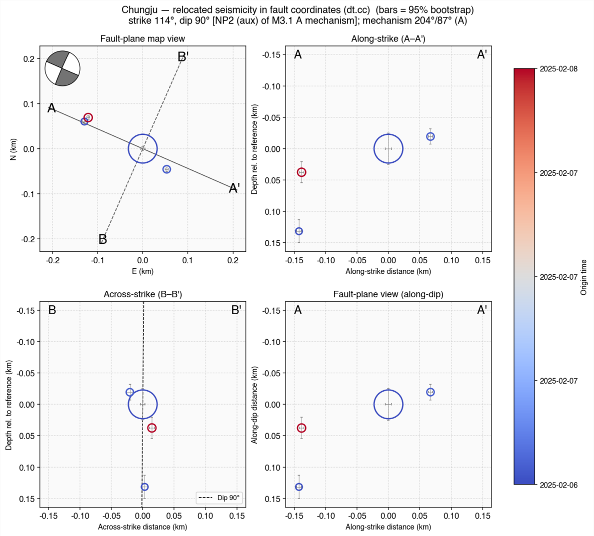
Output 4 — Waveform similarity (per dt.cc sub-cluster)





For each relocated sub-cluster (hypoDD `cid`), at the station *nearest to and common to* its events: a **full-waveform**

Output 5 — Fault-coordinate sections (SVD frame, v1.14.0+)



The relocated cloud rotated into a fault frame: map view, along-strike and across-strike depth sections (dashed = dip), and the along-dip view.

Since **v1.14.0** the frame defaults to the **SVD best-fit plane of the cloud** (`FRAME_FROM = "svd"`) — a property of *the relocation*, constrained by every event, needing no focal mechanism. The beachball is still drawn and the title reports both planes.

Caveat. A near-equant cloud has a good *plane* but a weak *strike within it*. Check the singular values in section 5: 2026 Haenam gives 372/269/45 m — 6:1 flattening, but only 1.4:1 in-plane. Use `"auto"` or an explicit `strike=`/`dip=` there.

`FRAME_FROM` is a **notebook parameter, not a CLI flag**: edit the params cell of `03_results_<cluster>.ipynb`. *(Layout from chungju; it predates the SVD default.)*

Run on your own catalog

1. Save your KMA-style CSV (the format from slide 6).
2. One command:

```
./pocketquake.sh ~/my_catalog.csv myswarm
```

3. Override the auto-derived epicenter / bounds if your catalog spans a wide region:

```
./pocketquake.sh ~/my_catalog.csv myswarm \
    --epi 35.46,128.43 \
    --bounds 35.3,35.65,128.25,128.65
```

4. Mainshock treatment (re-runs xcorr→dt.cc with the mainshock excluded, builds a `_main` notebook):

```
./pocketquake.sh ~/my_catalog.csv myswarm \
    --mainshock 20240912144719
```

Adding new events later (`--augment`, v1.13.0+)

A sequence is still going, or the catalog grew. **Do not re-run the cluster from scratch** — add only what is new:

```
./pocketquake.sh ~/my_catalog_updated.csv myswarm --augment --dry-run
./pocketquake.sh ~/my_catalog_updated.csv myswarm --augment
```

Only the **new** events are downloaded, gathered, picked and located. Existing picks, SAC pick headers and per-pair `dt.cc` files are reused (`xcorr` computes only new-vs-all pairs); the *whole* cluster is then re-relocated.

Strictly additive aborts and lists any event in the run but missing from the new catalog. Drifted lat/lon/depth/mag are warned; original rows stay.

Idempotent interrupted runs resume; "nothing to add" exits clean; events the source cannot serve yet are skipped and retried next time.

Identity pinned `event_manifest.csv` freezes existing HypoDD cuspids and appends new ones, so cached pair files stay valid even when new events interleave in time with the old.

Uses the cluster's stored epicenter / bounds / source, so `--epi`/`--bounds`, `--mainshock` and the backend switches are rejected (apply those in a follow-up run).

When the download will not serve an event yet

Recent events sometimes are not on the NECIS API yet, but *are* downloadable by hand from the NECIS event-search cart. Stage that bulk ZIP, then augment offline:

```
python -m pocketquake.necis_zip ~/Downloads/422892.zip \
    --cluster myswarm --dry-run
python -m pocketquake.necis_zip ~/Downloads/422892.zip --cluster myswarm

./pocketquake.sh ~/my_catalog_updated.csv myswarm \
    --augment --skip-download
```

The outer ZIP holds one `<NECIS_ID>.{a,v}.zip` per event. Events are identified from the **origin-stamped miniSEED** names (`NET.STA.CHA.YYYY.DDD.HH.MM.SS`), which are identical across every trace of one event — never from SAC names, which carry per-station segment start times. Only events in the cluster catalog are staged, and by default only those whose waveforms are missing.

Command-line options (`./pocketquake.sh --help`)

`--python`	pure-Python backend (HypoSVI + relocDD-py; no Fortran)
`--compare`	run Fortran *and* Python on the same picks → side-by-side notebook
`--source necis | stp | mixed`	waveform source (default `necis`; use `stp`/`mixed` for **pre-2020** events)
`--skip-download`	reuse waveforms already on disk (skip the slow download)
`--skip-pipeline`	scaffold + download only (no relocation / notebook)
`--mainshock <UTC>`	add Gwangyang-style mainshock treatment (`_main` notebook)
`--augment`	add only the *new* catalog events to an existing cluster, reusing picks + dt.cc pairs (`--dry-run` previews the diff)
`--picker phasenet_plus | stead`	picker model (`stead` needs no EQNet)
`--velmodel kim1983 | kim2011`	velocity model for relocation + focal mech + notebook
`--cores N`	cap xcorr workers (default ∼10)
`--epi`, `--bounds`	override the auto-derived epicenter / region
`--fg`	foreground (default: background, logs to `<slug>_run.log`)

Velocity model: location always computes both kim1983 + kim2011; `--velmodel` (default kim1983) selects which drives the relocation + focal mechanisms + notebook. Fully for the Fortran path; with `--python`, HypoSVI keeps its bundled EikoNet weights.

Common pitfalls (`docs/TROUBLESHOOTING.md`)

`python interpreter not found` set `POCKETQUAKE_PYTHON=/path/to/python` in `.env` or put `python3` on `$PATH`.

`Permission denied (publickey)` use the HTTPS clone URL, not SSH.

`Submodule path '...' not registered` you forgot `-recurse-submodules`; fix with `git submodule update -init -recursive`.

`EQNET_DIR not set` PhaseNet+ is the default picker; either install EQNet (slide 5) or pass `-picker stead`.

`libstdc++ version not found` glibc/libstdc++ too old (CentOS 7); upgrade OS or run the NECIS download elsewhere.

`NECIS_USER empty` fill in `.env`.

Where to go next

- **Read the executed result notebook** — `external/korea-cluster-relocation/pipeline/notebooks/03_results_<slug>.ipynb`
- **Inspect the auto-derived configuration** — `external/korea-cluster-relocation/pipeline/clusters/<slug>.py`
- **Tune the run** — `./pocketquake.sh -help`
- **Other examples bundled** — `examples/sangju/`, `examples/chungju/`, the cluster modules registered in `korea-cluster-relocation/pipeline/clusters/`
- **Docs** — `docs/INSTALL.md`, `docs/EXTERNAL_TOOLS.md`, `docs/TROUBLESHOOTING.md`

https://github.com/seismoseo/PocketQuake

Made by Min-Seong Seo / SGTL, SNU.